



Bank Statement ETL Engine

Product Requirements Document

Document ID:	BSE-PRD-001
Version:	1.0
Date:	2026-06-21
Status:	Active — Living Document
Owner:	channa.meng

One click. Any bank. Any format. Clean Excel. Every time.

1. Executive Summary

This document defines the complete product requirements for the **Bank Statement ETL Engine** — a professional, standalone Python-based desktop system that reads bank statement files in any format (CSV, XLS, XLSX, PDF), normalises them into a clean standard structure, and produces organised Excel output files grouped by **bank / year / month / currency**.

Feature	Description
Single Click	One launcher.bat opens the entire application — no terminal, no configuration.
Any Format	Source files can be CSV, XLS, XLSX, or PDF . Output is always clean XLSX .
Standalone	Each bank engine runs in its own Python venv — completely independent.
Auto-Detect	Year, month, and currency are read from the data — never typed manually.
Scalable	Designed for ABA Bank first , expandable to 200+ banks with no refactoring.
GUI	Professional PyQt6 desktop interface — colourful, friendly for all skill levels.
Database	SQLite now. PostgreSQL via Docker when volume justifies it.

2. Project Vision & Goals

Vision: One professional desktop application that any user — from an absolute beginner to a power user — can open, select a bank, drop a statement file, and receive a clean, organised Excel output in seconds. No command line. No configuration. No manual column mapping at runtime.

#	Goal	Priority
G1	Process any format (CSV/XLS/XLSX/PDF) → clean XLSX	Must Have
G2	Single launcher.bat — one click opens everything	Must Have
G3	Each bank engine isolated in its own venv	Must Have
G4	Auto-detect year, month, currency from transaction data	Must Have
G5	Output folder auto-created: output/aba/2025/202501/USD/	Must Have
G6	Professional PyQt6 GUI — colourful, beginner-friendly	Must Have
G7	SQLite database to persist all processed transactions	Must Have
G8	Proof of concept with ABA Bank only first	Must Have
G9	Architecture supports 200 banks without refactoring	Must Have
G10	Future migration to PostgreSQL via Docker	Should Have

3. Core Design Principles

P1 — One Bank, One Engine, One Venv

Every bank has its own isolated Python virtual environment. Bank A's dependencies never affect Bank B.

P2 — Single Entry Point for the User

The user only ever sees and clicks ONE file: launcher.bat. The GUI handles everything else.

P3 — Auto-Detect, Never Ask

The engine figures out year, month, and currency by reading the transactions. The user is never asked.

P4 — Source Format is the Bank's Problem, Not the User's

Whether ABA sends a PDF or a CSV this month, the engine handles it silently. The user just drops the file.

P5 — Output is Always Clean XLSX

No matter what format comes in, what comes out is always a clean, formatted .xlsx Excel file.

P6 — Logic Stays Separate from Interface

ETL pipeline code must never know about the GUI. The GUI calls the pipeline via subprocess.

P7 — Each Bank is a Blueprint Copy

Adding Bank #50 means: copy the aba/ folder, rename it, fill in the column mapping. Nothing else changes.

4. System Architecture Overview

```

USER  double-clicks  launcher.bat
    ■
    ▼
launcher.bat → detects Python → launches  gui/main.py
    ■
    ▼
PyQt6 GUI
  ■■ Bank Selector      (auto-discovers banks/ folder)
  ■■ File Drop Zone     (drag & drop or Browse button)
  ■■ Process Button     (triggers the selected bank's engine)
  ■■ Progress Display   (Extracting... Transforming... Exporting...)
  ■■ Results Table      (shows all processed transactions)
    ■
    ■ calls via subprocess
    ▼
Per-Bank Standalone Engine
  banks/aba/run.py ← called with source file path as argument
    ■■ detect.py        → identify CSV / XLS / XLSX / PDF
    ■■ extract.py       → read into DataFrame
    ■■ transform.py     → map columns to standard schema
    ■■ export.py        → write clean .xlsx output
    ■
    ▼
output/aba/2025/202501/USD/aba_202501_USD.xlsx
    ■
    ▼
database/bank_statements.db  (SQLite Phase 1)
PostgreSQL via Docker        (Phase 2 – future)
  
```

5. Folder Structure

```

project-root/
  ■
  ■■■ launcher.bat      ← THE ONLY FILE THE USER CLICKS
  
```

```

■
■■■ gui/                                ← Shared PyQt6 GUI application
■   ■■■ venv/                          ← GUI's own virtual environment
■   ■■■ requirements.txt                ← PyQt6 + GUI dependencies only
■   ■■■ main.py                        ← Entry point
■   ■■■ windows/                      ← Window classes
■   ■■■ widgets/                      ← Reusable UI components
■   ■■■ assets/styles/theme.qss       ← Color theme stylesheet
■
■■■ banks/
■   ■■■ aba/                          ← ABA Bank – fully standalone
■   ■   ■■■ venv/                     ← ABA's own venv (.gitignored)
■   ■   ■■■ requirements.txt          ← ABA's own dependencies
■   ■   ■■■ setup.bat                 ← Creates venv and installs deps
■   ■   ■■■ run.py                    ← Entry point called by GUI
■   ■   ■■■ config.py                 ← ABA column mappings
■   ■   ■■■ pipeline/
■   ■       ■■■ detect.py
■   ■       ■■■ extract.py
■   ■       ■■■ transform.py
■   ■       ■■■ export.py
■   ■   ■■■ db/
■   ■       ■■■ sqlite_loader.py
■   ■
■   ■■■ [future banks follow same structure]
■
■■■ downloads/
■   ■■■ aba/                          ← Drop ABA source files here
■
■■■ output/
■   ■■■ aba/
■       ■■■ 2025/
■           ■■■ 202501/
■               ■■■ USD/
■                   ■■■ aba_202501_USD.xlsx
■                   ■■■ KHR/

```

- ■■■ aba_202501_KHR.xlsx
-
- database/
 - ■■■ bank_statements.db ← SQLite database
-
- PRD.md ← This document

6. Single Launcher — launcher.bat

The user only ever sees **one file: launcher.bat**. This is a fundamental UX requirement. One double-click launches the entire system. There must never be one .bat file per bank — that would force users to know which bank to click, and scale to 200 .bat files.

```
launcher.bat (simplified logic)

1. Check Python 3.10+ is installed
2. If gui/venv does not exist → run gui/setup.bat to create it
3. For each bank in banks/ → if venv missing → run banks/{name}/setup.bat
4. Launch: gui/venv/Scripts/python.exe gui/main.py
5. GUI takes full control – no terminal visible to user
```

The GUI calls each bank engine programmatically via **subprocess** — passing the source file path as an argument and reading a JSON result from stdout. The user selects the bank in the GUI dropdown; the launcher never needs to know which bank.

7. ETL Pipeline — Extract Transform Load

Stage	Input	Output	Key Logic
DETECT	File path	file_type, valid flag	Read extension + magic bytes
EXTRACT	File path + type	Raw DataFrame	pandas / xlrd / pdfplumber
TRANSFORM	Raw DataFrame	List of standard records	Apply COLUMN_MAP, cast types
EXPORT	Standard records	Clean .xlsx file	Auto-create folder, format table
LOAD	Standard records	SQLite rows	INSERT OR IGNORE (no duplicates)

COLUMN_MAP — How Each Bank Teaches the Engine

Each bank's **config.py** contains a **COLUMN_MAP** dictionary. The keys are the bank's actual column header names exactly as they appear in the source file. The values are the standard field names that every bank's output must conform to.

```
# banks/aba/config.py (example – to be confirmed with real ABA statement)

BANK_NAME = "aba"

COLUMN_MAP = {
    "Tran Date" : "transaction_date",
    "Narration" : "description",
    "Debit" : "debit_amount",
    "Credit" : "credit_amount",
```

```
"Balance"      : "balance",  
"Ref No"       : "reference_id",  
"CCY"          : "currency",  
"Account No"   : "account_number",  
}
```

8. Standard Output Schema

Every bank, regardless of source format, produces records conforming to this schema. This is the contract that the ETL pipeline, the Excel exporter, and the database all rely on.

Field	Type	Required	Description
transaction_date	date	Yes	Date the transaction occurred
account_number	str	No	Bank account number
account_name	str	No	Account holder name
reference_id	str	No	Bank's transaction reference
description	str	No	Narration / transaction description
debit_amount	Decimal	No	Money going out (always positive)
credit_amount	Decimal	No	Money coming in (always positive)
balance	Decimal	No	Running balance after transaction
currency	str	Yes	3-letter code: USD, KHR, etc.
bank_name	str	Yes	Bank identifier e.g. 'aba'
source_file	str	Yes	Original filename processed

Amount convention: Debit and Credit are always stored as **positive numbers** in separate columns. If the bank uses a single amount column with +/- signs, the transform stage splits them.

9. Output File & Folder Convention

```
output/  
  {bank_name}/  
    {year}/  
      {year}{month_2digits}/  
        {CURRENCY}/  
          {bank_name}_{year}{month_2digits}_{CURRENCY}.xlsx
```

Examples:

```
output/aba/2025/202501/USD/aba_202501_USD.xlsx  
output/aba/2025/202501/KHR/aba_202501_KHR.xlsx  
output/aba/2025/202502/USD/aba_202502_USD.xlsx  
output/aba/2024/202412/USD/aba_202412_USD.xlsx
```

What to determine	How it is determined
-------------------	----------------------

Year + Month	Most frequent (year, month) pair from transaction_date values
Currency	Most frequent currency value across all records
Multi-currency	If USD and KHR both appear → produce two separate output files

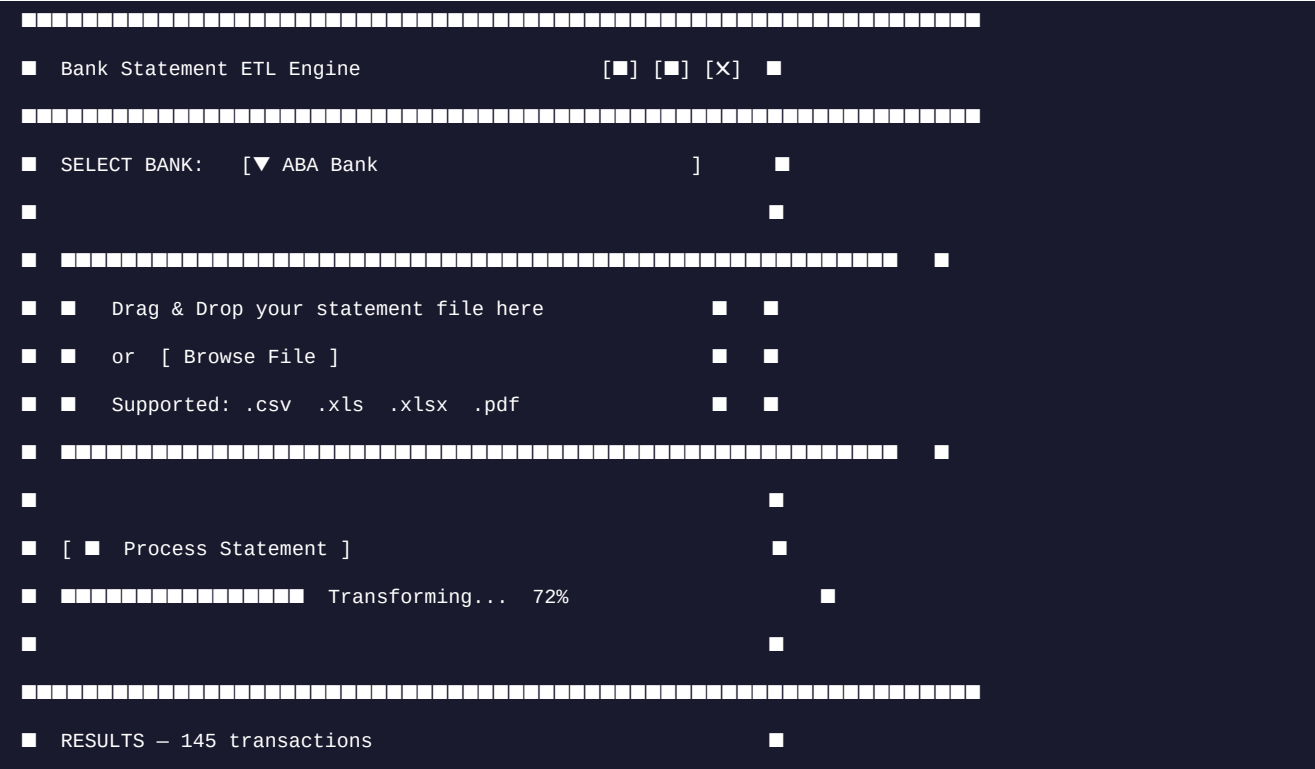
10. GUI Specification — PyQt6

The GUI is built with **PyQt6**. It must work for an absolute beginner. Every action must have a clear visual result. The terminal must never be visible to the user.

Color Theme

Element	Color	Hex Code
Window background	Dark Navy	#1a1a2e
Panel background	Deep Blue	#16213e
Buttons / Accent	Royal Blue	#0f3460
Button hover	Crimson Red	#e94560
Success indicator	Emerald Green	#00b894
Error indicator	Coral Red	#e17055
Gold accent bar	Amber Gold	#f0a500
Table header	Royal Blue	#0f3460
Text primary	White	#ffffff
Text secondary	Slate Gray	#a0aec0

Main Window Layout



[illegible]

11. Database Strategy

Phase	Database	Status	When
Phase 1	SQLite	Active	Now — single file, no server needed
Phase 2	PostgreSQL	Future	When volume or multi-user access requires it

```
-- SQLite schema (database/bank_statements.db)

CREATE TABLE IF NOT EXISTS transactions (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    bank_name     TEXT      NOT NULL,
    account_number TEXT,
    account_name  TEXT,
    transaction_date DATE    NOT NULL,
    reference_id  TEXT,
    description   TEXT,
    debit_amount  REAL,
    credit_amount REAL,
    balance       REAL,
    currency      TEXT      NOT NULL,
    source_file   TEXT,
    imported_at   DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(bank_name, transaction_date, reference_id, debit_amount, credit_amount)
);

-- INSERT OR IGNORE → re-processing same file never creates duplicates
```

12. Bank ABA — Proof of Concept

Property	Value
Bank Name	ABA Bank
Country	Cambodia
Engine ID	aba
Source Folder	downloads/aba/
Output Folder	output/aba/
Common Currencies	USD, KHR

Status	Phase 1 — Active Development
--------	------------------------------

Before the ABA engine is built, the following must be confirmed by reviewing a real ABA bank statement file:

- Exact column headers as they appear in the source file
- File format ABA provides (CSV, XLSX, PDF — or multiple formats)
- Date format used (e.g. 01/01/2025 or 2025-01-01)
- Amount format: single column +/- or separate Debit/Credit columns
- Which row the data starts from (any header/summary rows to skip?)
- If XLSX: which sheet name contains the transaction data
- If PDF: does the column header repeat on every page?

13. Scalability Plan — Up to 200 Banks

Adding a new bank requires editing **exactly one file** — the bank's own **config.py**. All other files are copied unchanged from the ABA template. The GUI, launcher, database, and output structure require zero changes.

How to add Bank #2 (and every bank after that):

Step 1 — Copy the template

```
cp -r banks/aba/ banks/newbank/
```

Step 2 — Edit only config.py with the new bank's column names

```
BANK_NAME = "newbank"

COLUMN_MAP = {
    "TransDate" : "transaction_date",
    "Details"    : "description",
    "DR"         : "debit_amount",
    "CR"         : "credit_amount",
    ...
}
```

Step 3 — Create the downloads folder

```
mkdir downloads/newbank/
```

Step 4 — Done.

The GUI auto-discovers the new bank and shows it in the dropdown.

No changes to launcher.bat, gui/, other banks, or the database.

14. Development Roadmap

Phase	Deliverable	Status
Phase 1	ABA standalone ETL engine (no GUI)	In Progress
Phase 1	SQLite loader and schema	Pending
Phase 1	launcher.bat (CLI mode, no GUI yet)	Pending
Phase 1	Test with real ABA statement files	Pending
Phase 2	PyQt6 GUI — main window and bank selector	Pending
Phase 2	File drop zone + process button	Pending
Phase 2	Results table + open folder button	Pending
Phase 2	Color theme applied	Pending
Phase 3	PyInstaller packaged executable	Pending
Phase 4	Second bank added (pattern proof)	Pending
Phase 4	PostgreSQL / Docker planning	Pending
Phase 5	Banks 3 through N	Future
Phase 5	Reporting features in GUI	Future

15. Glossary

Term	Definition
ETL	Extract, Transform, Load — the three-stage data processing pipeline
Source File	Raw bank statement file received from the bank (any format)
COLUMN_MAP	Dictionary in config.py mapping bank column names → standard names
Venv	Python virtual environment — isolated Python + packages per bank
XLSX	Microsoft Excel Open XML format — the output format for all statements
KHR	Cambodian Riel — the local currency of Cambodia
SQLite	Lightweight, serverless, file-based SQL database (Phase 1)
PostgreSQL	Full-featured SQL database server, run in Docker (Phase 2)
PyQt6	Python library for building professional desktop GUI applications
subprocess	Python mechanism to run one program from inside another
Upsert	Insert a record if it does not exist; skip if it already exists

Drop Zone	GUI area where you drag and drop a file to process it
launcher.bat	The single Windows batch file the user double-clicks to start the app

*This document is the authoritative reference for all design and implementation decisions. When in doubt about any aspect of the system, refer to this document. When a decision is made that contradicts or extends this document, update this document first. **Last Updated: 2026-06-21***